

CSE445 Lecture 6

k-Nearest Neighbors (kNN)

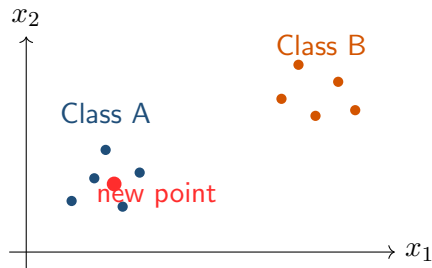
M.A.K., ECE, NSU

The Main Idea of Nearest Neighbor

If it looks like a duck and behaves like a duck, then it is probably a duck. In machine learning terms, this means: Similar inputs should have similar outputs.

So, when a new data point arrives, we compare it with the training examples. If it is closest to examples from one class, we assign it to that class.

Therefore, nearest neighbor is a similarity-based method: it predicts by looking at nearby examples in feature space.



Training Data and Test Data

Assume a training dataset with n examples and d features:

$$X \in \mathbb{R}^{n \times d}, \quad y = [y_1, y_2, \dots, y_n]^T$$

Each row $x_i \in \mathbb{R}^d$ is one training example, and y_i is its known label or target value.

Example	Feature 1	Feature 2	Feature 3	Label
x_1	2.1	5.0	1.2	A
x_2	1.8	4.7	1.0	A
x_3	6.2	2.4	5.8	B
x_4	5.9	2.0	6.1	B

For a new point x_* , kNN asks: which training examples are closest to x_* ?

What Does kNN Learn?

kNN is an instance based learning algorithm. It does not need any training or learning. It is a lazy learner algorithm. It finds the closest label at prediction time. No explicit learning is required.

Model type	What happens during training?
Parametric model kNN	Learns parameters such as weights w and bias b . Stores the training examples and waits until prediction time.

Because most of the work happens during prediction, kNN is called:

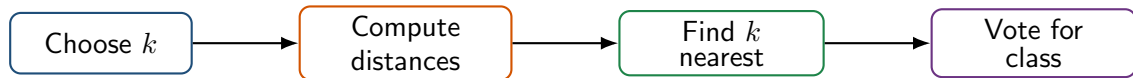
Instance-based Learning

Lazy Learning

This makes kNN easy to understand, but prediction can become slow when the dataset is large.

The kNN Classification Algorithm

For a new input x_* , kNN classification follows four basic steps.



Mathematically, after finding the neighbor set $\mathcal{N}_k(x_*)$, the predicted class is the most common label among those neighbors:

$$\hat{y}_* = \text{majority vote}\{y_i : x_i \in \mathcal{N}_k(x_*)\}.$$

The symbol $\hat{y}_*$ means the model's predicted output for the new point.

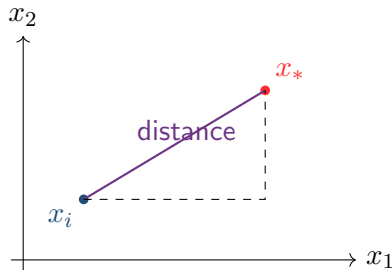
Measuring Distance: The Core of kNN

kNN depends heavily on the semantics of **Distance**. If distance is poorly chosen, the nearest neighbors may not be meaningful. or two data points

$$x_i = [x_{i1}, x_{i2}, \dots, x_{id}], \quad x_* = [x_{*1}, x_{*2}, \dots, x_{*d}],$$

The Euclidean distance is

$$d(x_i, x_*) = \sqrt{\sum_{j=1}^d (x_{ij} - x_{*j})^2}.$$



Common Distance Metrics

Different distance metrics define “near” in different ways.

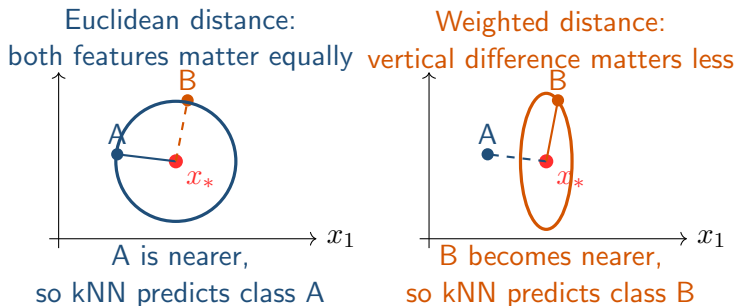
Metric	Formula	Useful idea
Euclidean	$\sqrt{\sum_j (x_j - z_j)^2}$	Straight-line distance
Manhattan	$\sum_j x_j - z_j $	Grid-like movement
Minkowski	$\left(\sum_j x_j - z_j ^p\right)^{1/p}$	General family of distances

For introductory ML, Euclidean distance is the most common starting point.

Important: Changing the distance metric can change the prediction.

Why the Distance Function Matters

In kNN, the distance function defines what **similar** means. Changing the distance function can change the selected neighbors and therefore change the prediction.



A distance function should reflect the meaning of the features. For example, **10 years of age** and **10 mg/dL glucose** should not be treated as equally important just because both numbers are 10.

Choosing a Meaningful Distance Function

A good distance function should reflect the structure of the data.

Data type	Reasonable distance idea
Numeric features	Euclidean or Manhattan distance after scaling features.
Text data	Compare word vectors or TF-IDF vectors using cosine similarity.
Binary features	Count how many entries are different, such as Hamming distance.
Images	Raw pixel distance is simple, but learned image features are often more meaningful.

Important lesson: distance should measure **semantic closeness**, not only numerical closeness.

For example, in house-price prediction, a difference of **one bedroom** may matter more than a difference of **one meter** in location coordinates. The distance function should represent this practical meaning.

Example: Find the Nearest Neighbor

Suppose the new point is $x_* = (3, 3)$. We compare it with three training points.

Point	Coordinates	Class	Distance to x_*
x_1	(2, 2)	A	$\sqrt{(2-3)^2 + (2-3)^2} = 1.41$
x_2	(4, 3)	B	$\sqrt{(4-3)^2 + (3-3)^2} = 1.00$
x_3	(6, 6)	B	$\sqrt{(6-3)^2 + (6-3)^2} = 4.24$

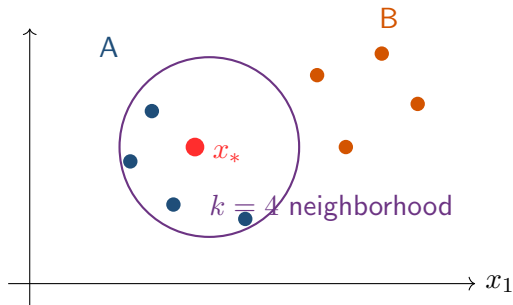
For $k = 1$, the nearest point is x_2 , so

$$\hat{y}_* = B.$$

For $k = 3$, the neighbors are A, B, and B, so majority voting still predicts B.

Visual Example: Majority Vote

In classification, kNN predicts the class that appears most often among the nearest neighbors.

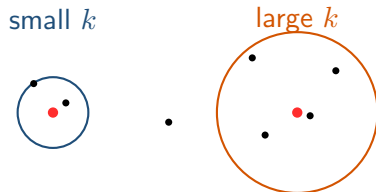


If the nearest neighbors contain more blue points than orange points, the prediction becomes **Class A**.

Choosing the Value of k

The hyperparameter k controls how local or smooth the decision is.

Choice of k	Effect
Small k	Very local; can fit noise; lower bias but higher variance.
Large k	Smoother decision; more stable; higher bias but lower variance.



In practice, choose k using a validation set or cross-validation.

Distance-Weighted Voting

Majority voting treats all selected neighbors equally. But sometimes closer neighbors should count more.

A common distance weight is

$$w_i = \frac{1}{d(x_i, x_*) + \epsilon},$$

where ϵ is a tiny positive number that avoids division by zero.

The predicted class is the class with the largest total weight:

$$\hat{y}_* = \arg \max_c \sum_{i \in \mathcal{N}_k(x_*)} w_i \mathbf{1}(y_i = c).$$

Meaning: a very close neighbor has a stronger vote than a far neighbor.

kNN Regression

kNN can also predict a **continuous value**, not only a class label.

For regression, after finding the k nearest neighbors, predict the average of their target values:

$$\hat{y}_* = \frac{1}{k} \sum_{i \in \mathcal{N}_k(x_*)} y_i.$$

Neighbor	Distance	Target value
x_1	1.0	80
x_2	1.5	86
x_3	2.0	92

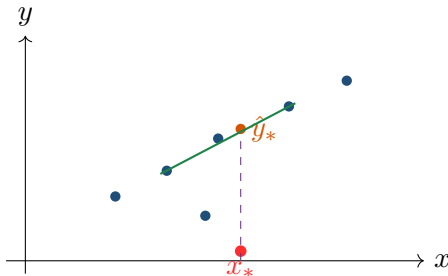
For $k = 3$:

$$\hat{y}_* = \frac{80 + 86 + 92}{3} = 86.$$

Weighted kNN Regression

In weighted regression, closer neighbors receive larger weights.

$$\hat{y}_* = \frac{\sum_{i \in \mathcal{N}_k(x_*)} w_i y_i}{\sum_{i \in \mathcal{N}_k(x_*)} w_i}$$



This makes the prediction more influenced by the most similar training examples.

Why Feature Scaling Matters

Distance can be dominated by features with large numerical ranges.

Feature	Typical scale	Problem
Age	18–70	moderate numbers
Income	10,000–200,000	very large numbers

If we use Euclidean distance directly, income may dominate the distance calculation. A common fix is **standardization**:

$$z_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j},$$

where μ_j and σ_j are the mean and standard deviation of feature j .

Rule: scale numeric features before using kNN unless all features already have comparable units.

Why Feature Selection Matters

kNN compares examples using **distance in feature space**. If a feature is irrelevant, it adds noise to the distance and can make truly similar points look far apart.

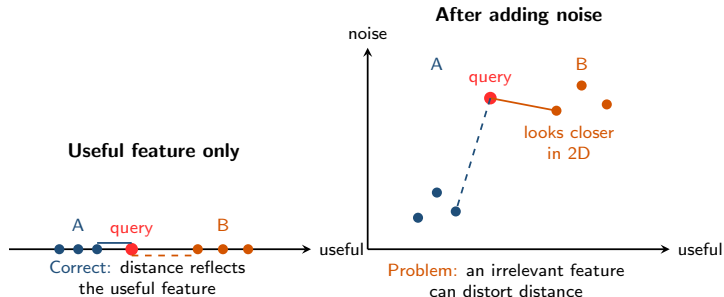
Good feature

separates classes in a meaningful way.

Irrelevant feature

changes the distance but does not help classification.

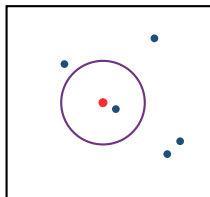
So, in kNN, **more features are not always better**. We want features that capture **real similarity**, not random variation.



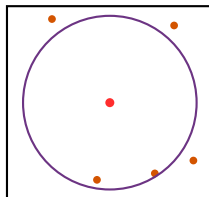
kNN and the Curse of Dimensionality

In high-dimensional spaces, distances become less informative. Points can become spread out so much that the nearest point is not much closer than an average point.

2D intuition



high-dimensional issue



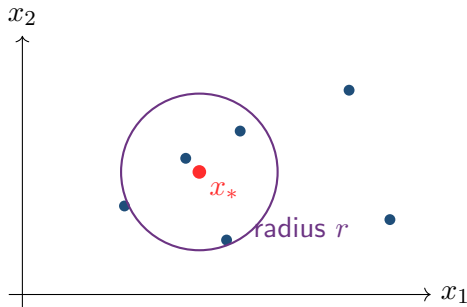
This is one reason kNN can struggle with very high-dimensional data such as raw images or text vectors.

Radius Nearest Neighbor

Radius nearest neighbor uses a fixed radius instead of a fixed number k .

$$\mathcal{N}_r(x_*) = \{x_i : d(x_i, x_*) \leq r\}$$

All points inside the radius become neighbors.



kNN vs Radius Neighbors

Both methods use distance, but they choose neighbors differently.

Method	Neighbor rule	Possible issue
kNN	Always chooses exactly k neighbors.	In sparse regions, neighbors may be far away.
Radius NN	Chooses all points within radius r .	May find no neighbors if r is too small.

Radius methods can adapt to dense and sparse regions, but choosing a good radius is important.

Computational Cost

A simple kNN implementation compares the new point with every training point.
For one test point:

$$\text{cost} \approx O(nd)$$

where n is the number of training examples and d is the number of features.



For large datasets, faster search structures or approximate nearest neighbor methods may be used.

Strengths of kNN

kNN is useful because it is simple, intuitive, and strong for some problems.

Strength	Why it matters
Easy to explain	Prediction is based on nearby examples.
No complex training	The algorithm mainly stores the training data.
Flexible decision boundary	Can model nonlinear class shapes.
Works for classification and regression	Same neighbor idea can predict labels or numeric values

Limitations of kNN

kNN also has important weaknesses.

Limitation	Meaning
Slow prediction	New points must be compared with many training points.
Sensitive to scaling	Large-scale features can dominate the distance.
Sensitive to irrelevant features	Noisy features can corrupt neighbor search.
Weak in high dimensions	Distance may lose meaning.
Choice of k matters	Too small or too large k can hurt performance.

Practical kNN Checklist

Before using kNN, check these steps.

- ① Choose useful numeric features.
- ② Scale the features, usually with standardization.
- ③ Choose a distance metric, often Euclidean distance.
- ④ Select k using a validation set or cross-validation.
- ⑤ Decide whether to use majority voting or distance-weighted voting.
- ⑥ Evaluate the model on unseen test data.

Good kNN performance depends on meaningful similarity.

Summary

k-nearest neighbors predicts a new point by looking at nearby training examples.

Concept	Key idea
Neighbor	A training point close to the new point.
Distance	Measures closeness between feature vectors.
k	Number of neighbors used for prediction.
Classification	Use voting among neighbor labels.
Regression	Use average or weighted average of neighbor targets.

The core assumption is simple: **nearby examples should have similar outputs.**